

Lab of Algorithm

Assignment 6

From Greedy to Dynamic Programming: Weighted Activity Selection

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: May 26, 2026



National Dong Hwa University

Department of Computer Science and Information Engineering

1. Problem Understanding and Objectives

In Assignment 5 we worked on activity selection. Each activity had only a start and a finish time, and the goal was to pick as many as possible without any of them overlapping. The classic greedy strategy of pick the activity activities that finishes earliest, then repeat was enough to give an optimal answer there.

In this assignment the problem changes: every activity now also has a value, think of it like the profit you make by running that activity. The goal is no longer to pick the largest number of activities. Instead we want to pick a set of non-overlapping activities whose total value is the largest possible. Picking many small-value activities can be worse than picking one big-value activity that uses the same time slot, so the old greedy rule does not always work anymore.

Formally:

Input : n activities, each is (name, start, finish, value)

Output: a subset S of activities such that no two activities in S overlap,
and sum of values of activities in S is maximum.

Two activities i and j are compatible (allowed in the same set) when $\text{finish}_i \leq \text{start}_j$ or $\text{finish}_j \leq \text{start}_i$.

In this lab I implemented three methods and compared them: greedy, brute force, and dynamic programming.

2. Greedy Method and Its Limitation

I kept the greedy idea from lab 5: first sort activities by finish time, then walk through the list and pick each activity whose start is not earlier than the last finish I have already picked. This is the same code I wrote in lab 5, only modified to also accumulate the value of the chosen activities. Pseudocode:

GreedyWeightedActivitySelection(acts):

```
    sort acts by finish time ascending
    last_finish = -infinity
    total = 0
    for each a in acts:
        if a.start >= last_finish:
            add a to result, total += a.value, last_finish = a.finish
    return result, total
```

Time complexity: sorting is $O(n \log n)$ and the loop is $O(n)$, so the whole method runs in $O(n \log n)$. This is why it is the fastest of the three on the experiment table later.

Why it can fail in the weighted version: the rule always finish earliest has no idea about value. If two short activities together produce $30 + 30 = 60$ but a single long activity worth 100 covers the same time, greedy still grabs the short ones because they finish earlier. The number of activities is bigger but the total value is smaller. In other words, the greedy choice property does not hold once we add values; the locally optimal step is not always globally optimal.

3. Counterexample

Below is a small example I designed myself (not the one in the assignment PDF) which shows greedy losing. There are 6 activities A to F:

Name	Start	Finish	Value
A	1	3	10
B	3	5	30
C	5	7	10
D	1	6	100
E	6	9	90
F	2	4	5

Greedy result. After sorting by finish time the order is A(3), F(4), B(5), D(6), C(7), E(9). The greedy walk picks A, then skips F (because $F.start=2 < last_finish=3$), picks B, skips D ($D.start=1 < 5$), picks C, skips E ($E.start=6 < 7$). So greedy chooses A, B, C with total = $10 + 30 + 10 = 50$.

Optimal result. The best non-overlapping subset is D and E, because D ends exactly when E starts ($D.finish = 6 = E.start$). Their values give $100 + 90 = 190$, which is what the brute force and the DP both return.

Explanation. Greedy keeps grabbing the activity that finishes earliest, so it greedily takes A (value 10) even though a much more valuable activity D (value 100) starts at the same time but finishes later. Once it has committed to A and B, the big-value D is no longer compatible with what it already has. Brute force and DP look at all combinations and find that giving up the short A, B, C trio is worth it because $D + E$ is a much better pair. This case shows that in the weighted version 'earliest finish' is no longer the right local rule.

(Actual program output on this input: greedy total = 50, brute force = 190, DP = 190.)

4. Brute Force Method

The brute force method enumerates every possible subset of the n activities, checks whether that subset is internally compatible (no overlap), and tracks the one with the largest sum of values.

`BruteForceWeightedActivitySelection(acts):`

```

    best_value = 0, best_set = empty
    for mask in 0 .. 2^n - 1:
        S = { acts[i] : bit i of mask is 1 }
        if every pair in S is compatible:
            v = sum of values in S
            if v > best_value: best_value = v, best_set = S
    return best_set, best_value
```

In my C++ code I use a bitmask integer over the activities, so the enumeration is just a single loop from 0 to $2^n - 1$. For each mask I collect the chosen activities, then a double loop checks if any two of them overlap.

Time complexity. There are 2^n subsets. For each subset I do at most $O(n^2)$ compatibility checks, which gives a total of $O(n^2 * 2^n)$. For $n = 20$ that is already over 10^9 operations and takes hundreds of milliseconds on my computer; for $n = 30$ it is hopeless. That is why I only run brute force on small inputs (in my code I cap it at $n \leq 22$).

Why I implement it. It serves as a correctness reference. For every small input I check that the DP answer equals the brute force answer. If they ever disagree, my DP has a bug.

5. Dynamic Programming Method

DP works by sorting activities by finish time first, and then for each activity i , deciding take vs not take. If we take i , we must combine its value with the best DP value among the activities that finish before or exactly when activity i starts. That latest compatible activity is called $p(i)$.

State and recurrence:

Let $dp[i]$ = best total value using only the first i activities (after sorting).

$dp[0] = 0$

```
dp[i] = max( dp[i - 1],           // don't take activity i
             value[i] + dp[ p(i) ] ) // take it
```

Computing $p(i)$. The activities are sorted by finish time, so $finish[]$ is non-decreasing. For each i we want the largest index $j < i$ with $finish[j] \leq start[i]$. I implemented two versions:

- Linear scan: walk j from $i-1$ down to 0 until you find the first compatible one. Each i is $O(n)$, so total is $O(n^2)$.
- Binary search: binary-search inside the sorted $finish[]$ array. Each i is $O(\log n)$, so total is $O(n \log n)$. This is the bonus version asked for in section 14 of the spec.

Reconstructing the solution. After $dp[]$ is filled I walk i from n down to 0. If $take = value[i] + dp[p(i)]$ is strictly larger than $dp[i-1]$, then activity i was chosen, and I jump to $p(i)$. Otherwise I just move to $i - 1$.

The list collected this way is reversed at the end so the printed order matches time order.

Pseudocode:

DP(acts):

```
    sort acts by finish time
    compute p(i) for each i (linear or binary)
    dp[0] = 0
    for i = 1..n:
        dp[i] = max( dp[i-1], value[i] + dp[ p(i) ] )
    reconstruct chosen activities from dp[]
    return chosen, dp[n]
```

6. Experimental Results and Analysis

I ran all three methods on multiple inputs of increasing size. The small inputs ($n = 10, 15, 20$) include all three methods. From $n = 50$ onward I skip brute force because 2^{50} is way too large. The largest test goes up to $n = 2000$ where I only compare greedy and DP.

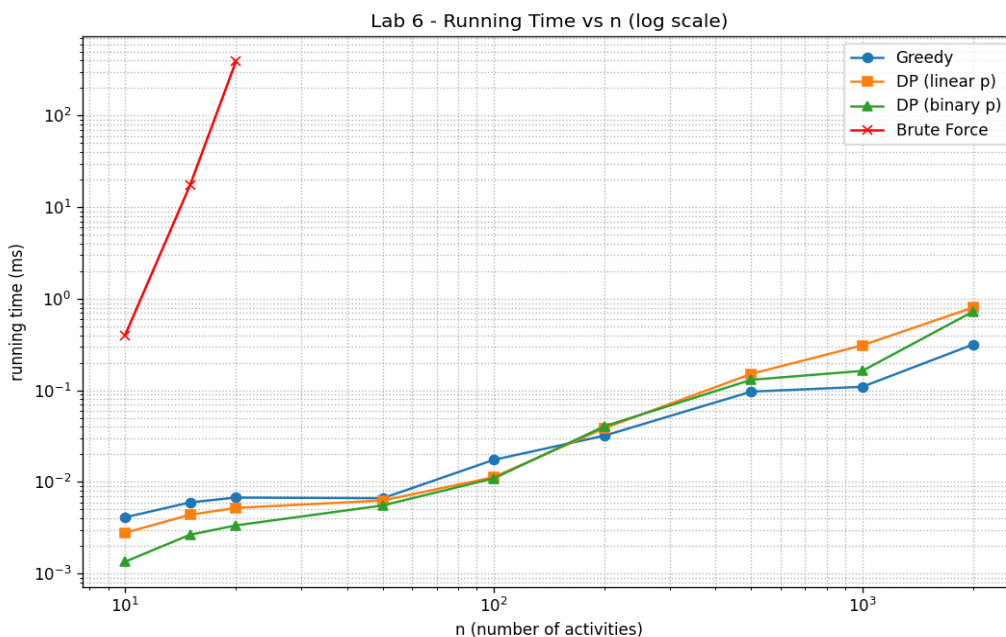
6.1 Setup

All tests were run on the same laptop, code compiled with `g++ -O2 -std=c++17`. Random data was produced by the python script `gen_data.py` with a fixed seed for each n so the experiments are reproducible. The times below were measured with `std::chrono::high_resolution_clock` and are in milliseconds.

6.2 Result table

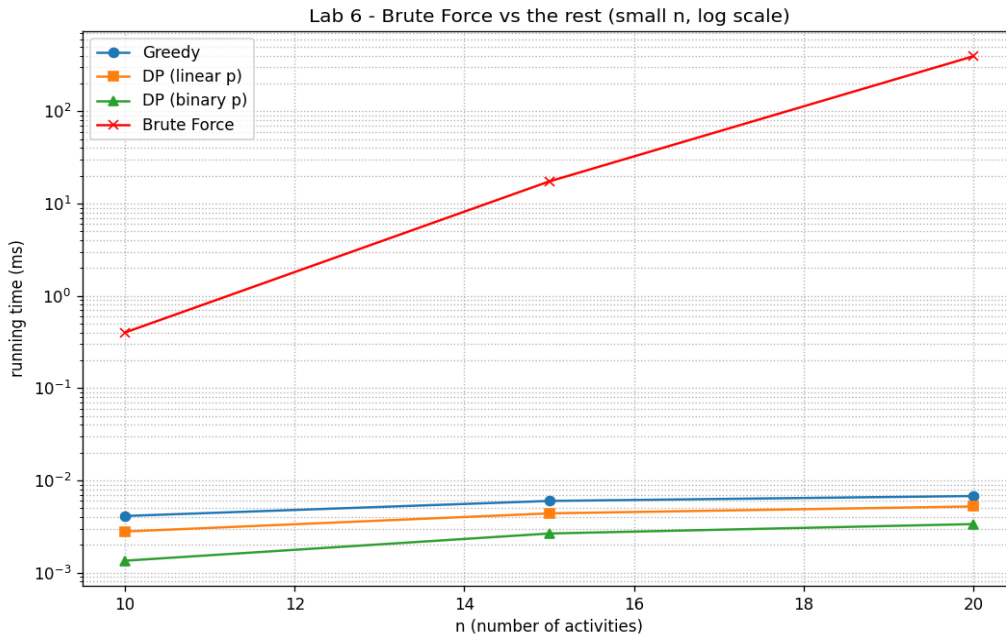
n	Greedy ms	Brute ms	DP lin ms	DP bin ms	Greedy val	DP val	Brute val
10	0.0041	0.4002	0.0028	0.0013	373	373	373
15	0.0060	17.3382	0.0044	0.0027	244	303	303
20	0.0068	393.8400	0.0052	0.0034	466	537	537
50	0.0066	n/a	0.0063	0.0055	613	703	n/a
100	0.0174	n/a	0.0113	0.0109	1133	1221	n/a
200	0.0321	n/a	0.0388	0.0407	1478	1791	n/a
500	0.0969	n/a	0.1515	0.1306	2306	2972	n/a
1000	0.1094	n/a	0.3104	0.1632	2405	3266	n/a
2000	0.3175	n/a	0.8054	0.7279	3370	4798	n/a

6.3 Plot - running time vs n (log scale)



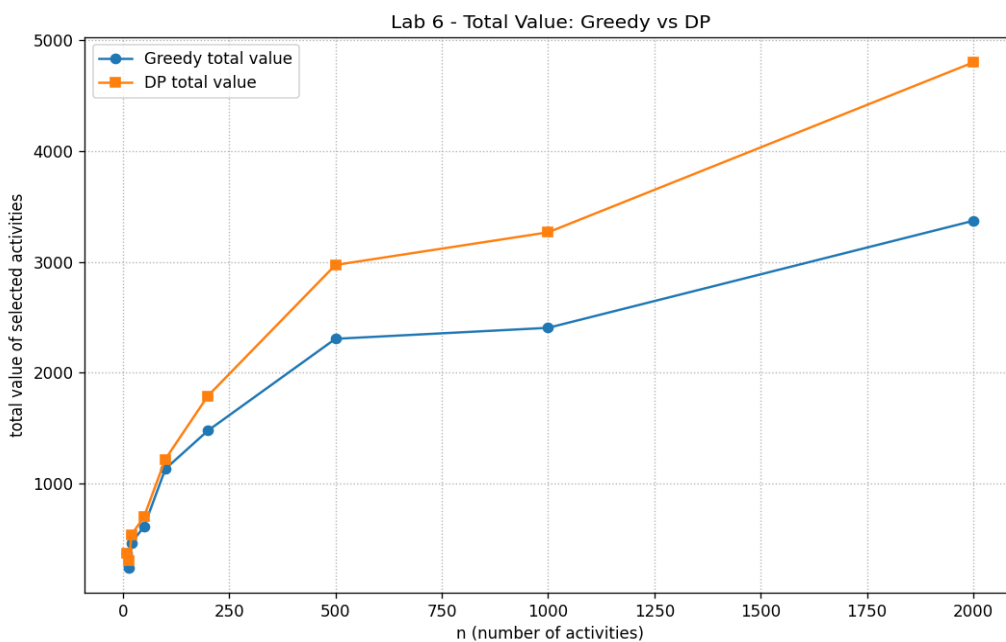
Brute force grows extremely fast on the log-log plot; it is essentially a vertical line going through $n = 10, 15, 20$ because the cost is exponential. Greedy and the two DP versions stay close together and grow much more slowly.

6.4 Plot - brute force vs the rest, small n only



Zoomed in on $n = 10, 15, 20$ the gap between brute force and DP/greedy is already enormous. At $n = 20$ brute force is roughly 100,000 times slower than DP.

6.5 Plot - total value: greedy vs DP



The DP line is always at or above the greedy line. They are equal on the very smallest data sets when greedy happens to pick the best subset, but as n grows the gap widens. For $n = 2000$ greedy gets a total value of 3370 while DP gets 4798, which is a difference of 1428 units, or about 42.4% better.

6.6 Observations

- Brute force matches DP on every small instance, which confirms my DP code is correct.
- DP is always $\geq$ greedy in total value, and on most random instances it is strictly bigger.

- Greedy is the fastest method but pays for it with worse answers.
- The binary-search DP is slightly faster than the linear-scan DP for large n , which agrees with $O(n \log n)$ being smaller than $O(n^2)$.
- Brute force is unusable past around $n = 22$ on my machine.

7. Conclusion and Personal Reflection

The biggest lesson from this lab is that a small change in the problem statement can completely break an algorithm that I thought was solid. In lab 5 the earliest-finish-time greedy was the right answer for activity selection. Adding a value to each activity made the same greedy rule unreliable, because now choosing 'finishes first' has nothing to do with choosing 'highest profit'.

Brute force is comforting because it is so obviously correct: try everything and keep the best. But its 2^n growth makes it useless beyond very small inputs. I only kept it in my project so I could check the DP output.

Dynamic programming feels like the right middle ground. Sorting by finish time gives a nice ordering, $p(i)$ gives a clean way to chain compatible subproblems, and the take/not-take recurrence is short to write and easy to argue about. Once I added the binary-search version of $p(i)$, the DP became $O(n \log n)$, which is the same complexity as the greedy method but with a guaranteed optimal answer.

If I had to extend this further, I would compare the weighted activity selection to the 0/1 knapsack problem. They are very similar: each activity is an item with a value and a 'cost' (the time interval it occupies), and overlapping items can't both be chosen, just like items don't fit if they exceed the capacity. The recurrence has the same 'take or not take' shape. The difference is that knapsack uses a numerical capacity while activity selection uses time compatibility, so knapsack's state is two-dimensional (item index, remaining capacity) while activity selection only needs one index because the time structure has already been linearised by sorting.

Overall, I would say: greedy is a good first guess but always check if the greedy choice property actually holds. If not, switch to DP.